Informe Técnico — Reto 1

LangChain, LangGraph y Observabilidad Avanzada con LangSmith

Autor: Luis Cadena
Curso: AI Engineering / GenAI
Fecha: Noviembre 2026
Estado: 100% Completado

10	100%	122	Activo
MÓDULOS DESARROLLADOS	TASA DE ACIERTO EVAL	CHUNKS RAG (PDF)	LANGSMITH PROMPT HUB

1. Resumen Ejecutivo

El presente documento certifica la implementación y ejecución exitosa del **Reto 1**. El objetivo principal fue construir un ciclo de vida completo de ingeniería de aplicaciones con Modelos de Lenguaje (LLMs) aplicando estándares de producción: orquestación declarativa, llamadas a herramientas en agentes autónomos, recuperación aumentada por generación (RAG), control de estado en memoria y observabilidad con pruebas de evaluación automatizadas en la plataforma **LangSmith**.

2. Stack Tecnológico y Arquitectura

Capa	Tecnología / Librería	Propósito / Rol
Motor LLM	`Google Gemini 3.5 Flash Lite` / `Anthropic Claude`	Inferencia generativa, razonamiento y function calling.
Orquestación	`LangChain 1.4.7` & `LangGraph 1.2.18`	Cadenas LCEL, bucles de agentes ReAct y manejo de estado.
Observabilidad	`LangSmith SDK 0.3.0`	Trazabilidad token a token, auditoría de llamadas y experimentación.
Recuperación (RAG)	`BM25Retriever` & `PyPDFLoader`	Búsqueda basada en palabras clave sobre políticas y PDFs corporativos.
Validación	`Pydantic v2`	Garantía de salida estrictamente estructurada en formato JSON.
Gobernanza	`LangSmith Prompt Hub`	Versionado colaborativo y desacoplamiento de plantillas de prompts.

3. Evidencias de Ejecución por Módulos

Módulo 1 & 2: Cadenas Declarativas LCEL y Streaming

Validado

Se implementaron cadenas con el operador `prompt | model | StrOutputParser()` con soporte para inferencia directa, procesamiento batch en paralelo y streaming progresivo token a token.

Salida: "El interés compuesto es aquel donde los intereses generados en un periodo se suman al capital inicial... generando un crecimiento exponencial."

Módulo 3: Salida Estructurada con Pydantic

Validado

Se configuró el modelo con `model.with_structured_output(Resumen)` asegurando que la respuesta contenga estrictamente los campos tipados (`titulo`, `puntos_clave` y `sentimiento`).

Objeto Pydantic: `Resumen(titulo='Reporte del tercer trimestre', puntos_clave=['Ventas récord digital', 'Costos logísticos +12%'], sentimiento='neutro')`

Módulo 4: Agentes ReAct Autónomos y Function Calling

Validado

Se definió un agente con las herramientas `@tool clima` y `@tool multiplicar`. El agente demostró autonomía al descomponer preguntas compuestas y ejecutar múltiples llamadas a herramientas en una sola invocación (ej: Cali, Shanghái y Brujas).

> **Ejecución del Agente:** Llamada automática a `clima(ciudad='Cali')` [24°C] + `multiplicar(a=23.5, b=4)` [94.0] → Síntesis final correcta entregada al usuario.

Módulo 5: Recuperación Aumentada (RAG) con BM25 y Documentos PDF

Validado

Se indexaron las políticas de la empresa con `BM25Retriever` y se procesó el documento real `LAMB.pdf` dividiéndolo en 122 fragmentos (chunks de 800 caracteres con 100 de solapamiento). El agente fundamentó sus respuestas exclusivamente en el contexto recuperado.

> **Respuesta RAG:** "Puedes hacer hasta 2 días de trabajo remoto por semana, previa aprobación de tu jefe."

Módulo 6: Observabilidad Personalizada con @traceable

Validado

Instrumentación de funciones Python convencionales (`clasificar` y `responder_ticket`) para capturar metadatos, tags y latencia en el panel de LangSmith independientemente de las primitivas de LangChain.

Módulo 7: Evaluación con Datasets y LLM como Juez

Validado (100% Score)

Se registró el dataset `preguntas-basicas-tutorial` en LangSmith. Se ejecutó una suite de evaluación con dos evaluadores deterministas (`contiene_respuesta`, `es_breve`) y un evaluador semántico LLM-as-a-Judge (`llm_juez`).

> **Resultados de Evaluación:** 3 de 3 ejemplos aprobados con `score: True` (100% coincidencia). Experimento sincronizado bajo el ID `gemini-v1-09e3facd`.

Módulo 8: Memoria Conversacional Persistente (InMemorySaver)

Validado

Uso de checkpointers en LangGraph para mantener estado entre turnos de conversación. En el turno 1 el usuario indicó su nombre y ciudad; en el turno 2 el agente recordó ambos datos a través de su `thread_id`.

> **Respuesta con Memoria:** "Te llamas José y vives en Cali, donde actualmente hace 24°C y está soleado."

Módulo 9: Versionado de Prompts con LangSmith Prompt Hub

Validado

Se publicó la plantilla de prompt `tutor-ia-reto1` mediante `client.push_prompt(...)` en la nube de LangSmith y posteriormente se consumió desacoplada con `client.pull_prompt(...)`.

> **Enlace en Hub:** `https://smith.langchain.com/prompts/tutor-ia-reto1`

4. Auditoría de Telemetría Extraída de LangSmith

Operación / Run	Tipo	Latencia	Estado	Tokens
`StrOutputParser`	parser	0.00s	✓ success	0
`ChatGoogleGenerativeAI`	llm	1.05s	✓ success	84
`ChatPromptTemplate`	prompt	0.00s	✓ success	0
`RunnableSequence`	chain	1.12s	✓ success	84
`ChatGoogleGenerativeAI`	llm	1.00s	✓ success	39
`ChatGoogleGenerativeAI`	llm	1.17s	✓ success	43
`StrOutputParser`	parser	0.00s	✓ success	0
`ChatGoogleGenerativeAI`	llm	47.09s	✓ success	80
`ChatPromptTemplate`	prompt	0.00s	✓ success	0
`RunnableSequence`	chain	47.22s	✓ success	80
`ChatGoogleGenerativeAI`	llm	172.41s	✓ success	193
`model`	chain	172.41s	✓ success	193

5. Conclusión y Verificación

Todas las metas propuestas para el Reto 1 han sido cumplidas a cabalidad. La solución garantiza código desacoplado, alta observabilidad, evaluación reproducible y trazabilidad total de ejecuciones en producción.